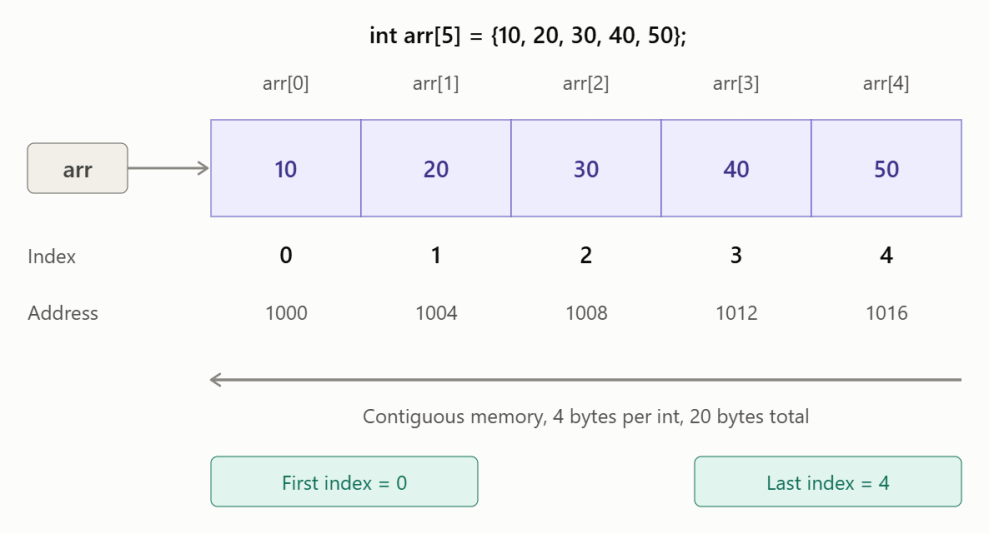
ARRAYS

- An array in C is a collection of similar (homogeneous) data type elements stored together under a single variable name with continuous memory location.
- Instead of creating separate variables for each value, you can use an array to store and manage them efficiently using index numbers.
- These index values are started from 0 to N-1 (N indicates size of an array).
- **Types of Arrays in C Language**
  - ✓ There are three types of arrays in C programming:
    1. One-Dimensional (or) Single Dimensional Arrays
    2. Two-Dimensional Arrays
    3. Multi-Dimensional Arrays

---

1. One-Dimensional Array:

❒ A one-dimensional array in C is the simplest form of an array. It stores a list of elements of the same data type in a single row (linear structure), accessed using a single index.



❒ ***Properties of One-Dimensional Array in C:***

The following are the important properties of a one-dimensional array in C:

- ✓ ***Homogeneous Data Type:*** All elements must be of the same data type (e.g., all int, all float, etc.).
- ✓ ***Fixed Size:*** The size of the array is defined at the time of declaration and cannot be changed later.
- ✓ ***Contiguous Memory Allocation:*** Array elements are stored in continuous memory locations, allowing fast access.

- ✓ ***Indexed Access:*** Elements are accessed using indices starting from 0 to n-1 (where n is array size).
- ✓ ***Random Access:*** Any element can be accessed directly using its index, without scanning other elements.
- ✓ ***Default Values Depend on Scope:*** Global/static arrays are initialized to 0 by default, local arrays contain garbage values unless explicitly initialized.
- ✓ ***Efficient Traversal with Loops:*** 1D arrays can be easily traversed using loops (for, while, etc.).
- ✓ ***Supports Compile-Time and Run-Time Initialization:*** You can assign values at the time of declaration or through input during execution.
- ✓ ***Cannot Store Different Data Types:*** Each element in a 1D array must be of the declared data type only.

❒ ***Declaration of One Dimensional Array in C***

- ✓ ***Basic Syntax***:

  <data_type> <array_name>[<array_size>];

  - data_type: Type of data (e.g., int, float, char)
  - array_name: Any valid identifier
  - array_size: Number of elements (must be a constant or macro)
- ✓ ***Examples for Declaration***:

```
int numbers[5];      // Integer array with 5 integers
float scores[10];    // Floating-point array with 10 floats
char letters[26];    // Character array with 26 characters

#define SIZE 3       // Using #define
int arr[SIZE];

const int n = 4;     // Using const variable
int data[n];
```

❒ ***Important Notes:***

- ✓ Array size must be a positive integer constant or a constant variable.
- ✓ Declaration alone doesn't assign values—elements will contain garbage values (if local) unless initialized.
- ✓ The array size is fixed and cannot be changed later in the program.

❒ ***Initialization of 1-Dimensional Array in C:***

- ✓ Array initialization means assigning values to the array elements either at the time of declaration or later using a loop.
- ✓ ***Compile-Time Initialization*** – initialization at the time of declaration and also called as static initialization.

```
int numbers[5] = {10, 20, 30, 40, 50};   // Full initialization
int values[5] = {1, 2};      // Partial initialization, it gives like {1,2,0,0,0}
int marks[] = {75, 80, 85};  // Initialization without specifying size
```

- ✓ ***Run-Time Initialization*** – initialization during program execution and it is also called as dynamic initialization.

```
int arr[3];                    // declaration
for (int i = 0; i < 3; i++) {
   scanf("%d", &arr[i]);       // reading 3 integers using loop
}
```

❒ ***Accessing Elements of 1D Array in C***

- ✓ Each element in a one-dimensional array is accessed using an index, which starts from 0 and goes up to (size - 1).
- ✓ ***Syntax:***

```
array_name[index];
```

array_name: Name of the array
index: Position of the element (0-based)

- ✓ ***Example:***

```
#include <stdio.h>
int main() {
   int marks[5] = {50, 60, 70, 80, 90};
   printf("First element: %d\n", marks[0]);          // 50
   printf("Third element: %d\n", marks[2]);          // 70
   marks[4] = 95;                          // Modifying the 5th element
   printf("Updated fifth element: %d\n", marks[4]);  // 95
   return 0;
}
```

Output:

```
First element: 50
Third element: 70
Updated fifth element: 95
```

- ✓ ***Example for Accessing All Elements Using Loop:***

```
#include <stdio.h>
int main() {
   int marks[5] = {50, 60, 70, 80, 90};   // compile-time initialization
   int i;
   for (i = 0; i < 5; i++) {
      printf("marks[%d] = %d\n", i, marks[i]);
   }
   return 0;
}
```

Output:

marks[0] = 50
marks[1] = 60
marks[2] = 70
marks[3] = 80
marks[4] = 90

- ***Memory Representation of One-Dimensional Array in C***
  - ✓ Memory representation of a one-dimensional array shows how array elements are stored in contiguous (continuous) memory locations in the system memory. Each element occupies memory based on its data type size.
  - ✓ ***Syntax:***

    Address of arr[i] = Base Address + (i × Size of Data Type)

    Where:

    Base Address = Address of the first element
    i = Index of the element
    Size of Data Type = Memory size of the array element (e.g., int = 4 bytes)

➢ Sample Example Programs for 1-Dimensional Arrays

- **Print All Elements of an Array**

```
#include<stdio.h>
void main(){
   int arr[10],i,n;
   //read array size
   printf("Enter Array Size:");
   scanf("%d",&n);
   //read array elements
   printf("Enter %d elements:",n);
   for(i=0;i<=n-1;i++)
      scanf("%d",&arr[i]);
   //display array elements
   printf("Display Array Elements:\n");
   for(i=0;i<=n-1;i++)
      printf("%d\t",arr[i]);
}
```

Output:

```
Enter Array Size:5
Enter 5 elements:12
16
28
45
```

```
6
Display Array Elements:
12    16    28    45    6
```

❒ **Sum of All Elements in Array**

```
#include<stdio.h>
void main(){
   int arr[10],i,n,sum=0;
   //read array size
   printf("Enter Array Size:");
   scanf("%d",&n);
   //read array elements
   printf("Enter %d elements:",n);
   for(i=0;i<=n-1;i++){
      scanf("%d",&arr[i]);
      sum=sum+arr[i];
   }
   //display the sum of array elements
   printf("Sum of array elements is %d",sum);
}
```

Output:

```
Enter Array Size:5
Enter 5 elements:3
2
4
5
1
Sum of array elements is 15
```

❒ **Find Maximum and Minimum Element in an Array**

```
#include<stdio.h>
void main(){
   int arr[10],n,i,min,max;
   // read array size
   printf("Enter array size:");
   scanf("%d",&n);
   // read array elements
   for(i=0;i<=n-1;i++)
      scanf("%d",&arr[i]);
   //calculating min and max values
   min=arr[0];
```

```
    max=arr[0];
    for(i=0;i<=n-1;i++){
      if(min>arr[i])
        min=arr[i];
      if(max<arr[i])
        max=arr[i];
    }
    //display max and min elements
    printf("Minimum element=%d \nMaximum element=%d",min,max);
}
```

Output:

```
Enter array size:5
12
16
28
45
6
Minimum element=6
Maximum element=45
```

- **Search for an Element in Array**

```
#include<stdio.h>
void main(){
  int arr[10],n,i,key,count=0;
  // read array size
  printf("Enter array size:");
  scanf("%d",&n);
  // read array elements
  for(i=0;i<=n-1;i++)
    scanf("%d",&arr[i]);
  //enter key element to search
  printf("Enter key element:");
  scanf("%d",&key);
  //search the element
  for(i=0;i<=n-1;i++){
    if(key==arr[i]){
      count++;
      break;
    }
  }
  if(count==0)
```

```
        printf("%d element not found in the array",key);
    else
        printf("%d element is found at position %d",key,i);
}
```

Output:

```
Enter array size:5
12
16
28
45
6
Enter key element:28
28 element is found at position 2
```

- **Reverse a One-Dimensional Array**

```
#include<stdio.h>
void main(){
    int arr[10],i,n;
    //read array size
    printf("Enter Array Size:");
    scanf("%d",&n);
    //read array elements
    printf("Enter %d elements:",n);
    for(i=0;i<=n-1;i++)
        scanf("%d",&arr[i]);
    //display array elements
    printf("Display Array Elements:\n");
    for(i=0;i<=n-1;i++)
        printf("%d\t",arr[i]);
    //display array in reverse
    printf("\n Reversed Array is:\n");
    for(i=n-1;i>=0;i--)
        printf("%d\t",arr[i]);
}
```

Output:

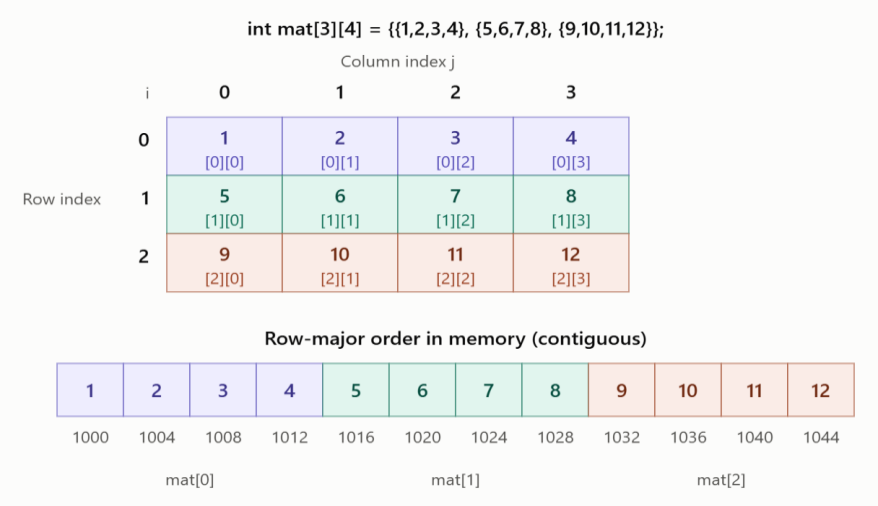
```
Enter Array Size:5
Enter 5 elements:12
16
28
45
```

6
Display Array Elements:
12 16 28 45 6
Reversed Array is:
6 45 28 16 12

---

2. Two-Dimensional Array:

- A two-dimensional array in C stores data in a table-like structure using rows and columns. It's often used to represent matrices.



- ***Declaration of Two Dimensional Array in C:***
  - ✓ ***Basic Syntax:***

    <data_type> <array_name>[<rows_size>][<columns_size>];

    - ***data_type*****:** Specifies the type of elements the array will store (like int, float, or char).
    - ***array_name:*** The name given to the array.
    - ***[rows_size]*** *and* ***[columns_size]****:* Define the size of the array, specifying how many rows and columns it contains.
  - ✓ ***Example for Declaration:***

    int matrix[3][4];

    - This example declares a two-dimensional array named matrix of type int with 3 rows and 4 columns. In total, this array can hold 3 x 4 = 12 integer values.
- ***Initialization of 2-Dimensional Array :***
  - ✓ Array initialization means assigning values to the array elements either at the time of declaration or later using a loop.

- ✓ ***Compile-Time Initialization*** – initialization at the time of declaration and also called as static initialization.
  - ***Syntax:***

```
<data_type> <array_name>[<rows_size>][<column_size>] = {
   {val1, val2, val3},
   {val4, val5, val6},
    ……
};
```

  - ***Example:***

```
int matrix[2][3] = {  {1, 2, 3},{4, 5, 6} };
```

- ✓ ***Run-Time Initialization*** – initialization during program execution and it is also called as dynamic initialization.

```
int matrix[2][3];              // declaration
int i,j;
for(i = 0; i < 2; i++) {
   for(j = 0; j < 3; j++) {
      scanf("%d",&matrix[i][j]);    // Reading 2D array values
   }
}
```

- ❒ ***Accessing Elements of 2D Array in C***
  - ✓ To access elements in a two-dimensional array in C, you use the row and column index numbers. The indexing always starts from 0.
  - ✓ ***Syntax:***

```
<array_name>[<row_index>][<column_index>];
```

    - ***row_index:*** Specifies the row position of the element.
    - ***column_index***: Specifies the column position of the element.
  - ✓ ***Example:***

```
int matrix[2][3] = {
   {10, 20, 30},
   {40, 50, 60}
};
```

    To access specific elements:
    - matrix[0][1] accesses the element in the first row, second column (20).
    - matrix[1][2] accesses the element in the second row, third column (60).
  - ✓ ***Example Program:***

```
#include <stdio.h>
int main() {
   int matrix[2][3] = {
     {10, 20, 30},
```

```
        {40, 50, 60}
    };
    printf("Element at [0][1]: %d\n", matrix[0][1]); // outputs 20
    printf("Element at [1][2]: %d\n", matrix[1][2]); // outputs 60
    return 0;
}
```

- ✓ ***Example for Accessing All Elements Using Loop:***

```
#include <stdio.h>
int main() {
    int matrix[2][3],i,j;
    printf("Enter 6 elements for 2x3 matrix:\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
   printf("\nMatrix elements are:\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");  // New line after each row
    }
    return 0;
}
```

Output:

```
Enter 6 elements for 2x3 matrix: 12
16
28
45
6
11
Matrix elements are:
12      16      28
45      6       11
```

- ❒ ***Memory Representation of Two-Dimensional Array in C***
  - ✓ Memory representation of a two-dimensional array shows how array elements are stored in contiguous (continuous) memory locations in the system memory. Each element occupies memory based on its data type size.
  - ✓ ***Syntax:***

Row-major order

Address of arr[i][j] = Base Address + ((i*C)+j) × Size of Data Type

(or)

Column-major order

Address of arr[i][j] = Base Address + ((j*R)+i) × Size of Data Type

Where:

Base Address = Address of the first element
i = row Index
j= column index
C=total number of columns (or) R= total number of rows
Size of Data Type = Memory size of the array element (e.g., int = 4 bytes)

- **Sample Example Programs for 2-Dimensional Arrays**
  - **Addition of Two Matrices**

```
#include<stdio.h>
void main(){
  int a[10][10],b[10][10],c[10][10],i,j,r1,c1,r2,c2;
  printf("Enter array-1 size:");
  scanf("%d%d",&r1,&c1);
  printf("\nEnter %dx%d size Array:",r1,c1);
  for(i=0;i<=r1-1;i++){
    for(j=0;j<=c1-1;j++){
      scanf("%d",&a[i][j]);
    }
  }
   printf("Enter array-2 size:");
  scanf("%d%d",&r2,&c2);
  printf("\nEnter %dx%d size Array:",r2,c2);
  for(i=0;i<=r2-1;i++){
    for(j=0;j<=c2-1;j++){
      scanf("%d",&b[i][j]);
    }
  }
  if(r1==r2 && c1==c2){
    for(i=0;i<=r1-1;i++){
      for(j=0;j<=c2-1;j++){
        c[i][j]=a[i][j]+b[i][j];
      }
    }
    printf("Addition of given Matrices is:\n");
    for(i=0;i<=r2-1;i++){
      for(j=0;j<=c1-1;j++){
```

```
            printf("%d\t",c[i][j]);
          }
          printf("\n");
        }
      }
      else
        printf("\nAddition is not possible....!");
    }
```

Output:

```
Enter array-1 size:2
2
Enter 2x2 size Array:1
4
2
3
Enter array-2 size:2
2
Enter 2x2 size Array:3
2
1
4
Addition of given Matrices is:
4     6
3     7
```

- **Transpose of a Matrix**

```
#include<stdio.h>
void main(){
  int arr[10][10],i,j,r,c;
  printf("Enter Array Size:");
  scanf("%d%d",&r,&c);
  printf("Enter %dx%d Array:",r,c);
  for(i=0;i<=r-1;i++)
    for(j=0;j<=c-1;j++)
      scanf("%d",&arr[i][j]);
  printf("Given Array is:\n");
  for(i=0;i<=r-1;i++){
    for(j=0;j<=c-1;j++)
      printf("%d\t",arr[i][j]);
    printf("\n");
  }
```

```
    printf("Transpose of Given Array:\n");
    for(i=0;i<=c-1;i++){
      for(j=0;j<=r-1;j++)
        printf("%d\t",arr[j][i]);
      printf("\n");
    }
}
```

Output:

```
Enter Array Size:2
3
Enter 2x3 Array:4
5
6
3
2
1
Given Array is:
4    5    6
3    2    1
Transpose of Given Array:
4    3
5    2
6    1
```

- **Multiplication of Two Matrices**

```
#include<stdio.h>
void main(){
  int a[10][10],b[10][10],c[10][10]={0},r1,c1,r2,c2,i,j,k;
  // Read matrix-1 size
  printf("Enter Array-1 Size:");
  scanf("%d%d",&r1,&c1);
  // Read matrix-2 size
  printf("Enter Array-2 Size:");
  scanf("%d%d",&r2,&c2);
  if(c1==r2){
    //Reading Matrix-1 elements
    printf("Enter matrix-1 with %dx%d size:\n",r1,c1);
    for(i=0;i<=r1-1;i++)
      for(j=0;j<=c1-1;j++)
        scanf("%d",&a[i][j]);
    //Reading Matrix-2 elements
```

```
        printf("Enter matrix-2 with %dx%d size:\n",r2,c2);
        for(i=0;i<=r2-1;i++)
           for(j=0;j<=c2-1;j++)
              scanf("%d",&b[i][j]);
        //perform matrix multiplication
        for(i=0;i<=r1-1;i++)
           for(j=0;j<=c2-1;j++)
              for(k=0;k<=c1-1;k++)
                 c[i][j]+=a[i][k]*b[k][j];
        //display matrix multiplication
        printf("Matrix Multiplication is:\n");
        for(i=0;i<=r1-1;i++){
           for(j=0;j<=c2-1;j++)
              printf("%d\t",c[i][j]);
           printf("\n");
        }
     }
     else
        printf("\n Matrix Multiplication is not Possible....!");

}
```

Output:

```
Enter Array-1 Size:2
3
Enter Array-2 Size:3
2
Enter matrix-1 with 2x3 size:
2
1
3
2
3
1
Enter matrix-2 with 3x2 size:
2
3
1
2
2
3
```

Matrix Multiplication is:
11 17
9 15

- **Subtraction of Two Matrices**

```
#include<stdio.h>
void main(){
  int a[10][10],b[10][10],c[10][10],i,j,r1,c1,r2,c2;
  printf("Enter array-1 size:");
  scanf("%d%d",&r1,&c1);
  printf("\nEnter %dx%d size Array:",r1,c1);
  for(i=0;i<=r1-1;i++){
    for(j=0;j<=c1-1;j++){
      scanf("%d",&a[i][j]);
    }
  }
   printf("Enter array-2 size:");
  scanf("%d%d",&r2,&c2);
  printf("\nEnter %dx%d size Array:",r2,c2);
  for(i=0;i<=r2-1;i++){
    for(j=0;j<=c2-1;j++){
      scanf("%d",&b[i][j]);
    }
  }
  if(r1==r2 && c1==c2){
    for(i=0;i<=r1-1;i++){
      for(j=0;j<=c2-1;j++){
        c[i][j]=a[i][j]-b[i][j];
      }
    }
    printf("Substraction of given Matrices is:\n");
    for(i=0;i<=r2-1;i++){
      for(j=0;j<=c1-1;j++){
        printf("%d\t",c[i][j]);
      }
      printf("\n");
    }
  }
  else
    printf("\n Substraction is not possible....!");
}
```

Output:

```
Enter array-1 size:2
2
Enter 2x2 size Array:3
4
2
1
Enter array-2 size:2
2
Enter 2x2 size Array:5
6
3
4
Substraction of given Matrices is:
-2     -2
-1     -3
```

- **Sum of Rows and Columns of a Matrix**

```
#include <stdio.h>
int main() {
   int matrix[10][10], rows, cols, i, j, sum;
   printf("Enter the number of rows and columns: ");
   scanf("%d %d", &rows, &cols);
   printf("Enter %dx%d matrix:\n",rows,cols);
   for (i = 0; i < rows; i++) {
      for (j = 0; j < cols; j++) {
         scanf("%d", &matrix[i][j]);
      }
   }
    printf("Given Array is:\n");
   for (i = 0; i < rows; i++) {
      for (j = 0; j < cols; j++) {
         printf("%d\t", matrix[i][j]);
      }
      printf("\n");
   }
   printf("\n--- Sum of Rows ---\n");
   for (i = 0; i < rows; i++) {
      sum = 0;
      for (j = 0; j < cols; j++) {
         sum += matrix[i][j];
```

```
        }
        printf("Sum of Row %d = %d\n", i + 1, sum);
    }
    printf("\n--- Sum of Columns ---\n");
    for (j = 0; j < cols; j++) {
        sum = 0;
        for (i = 0; i < rows; i++) {
            sum += matrix[i][j];
        }
        printf("Sum of Column %d = %d\n", j + 1, sum);
    }

    return 0;
}
```

Output:

```
Enter the number of rows and columns: 2
3
Enter 2x3 matrix:
4
2
5
1
3
6
Given Array is:
4    2    5
1    3    6

--- Sum of Rows ---
Sum of Row 1 = 11
Sum of Row 2 = 10

--- Sum of Columns ---
Sum of Column 1 = 5
Sum of Column 2 = 5
Sum of Column 3 = 11
```

- **Identity matrix pattern**

```
#include<stdio.h>
void main(){
    int arr[10][10],i,j,row,col;
```

```
    printf("Enter Array Size:");
    scanf("%d%d",&row,&col);
    printf("Identity Matrix is:\n",row,col);
    for(i=0;i<row;i++){
      for(j=0;j<col;j++){
        if(i==j)
          printf("1\t");
        else
          printf("0\t");
      }
      printf("\n");
    }
}
```

Output:

```
Enter Array Size:3
3
Identity Matrix is:
1    0    0
0    1    0
0    0    1
```

❒ **Print the alphabet triangle using a 2D char array**

A
A B
A B C

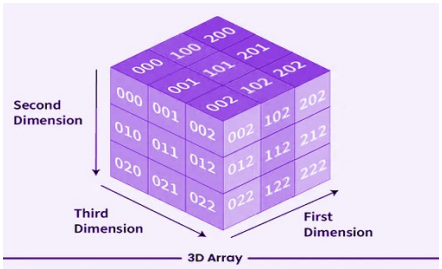
```
#include<stdio.h>
void main(){
  int i,j,n;
  char ch;
  printf("Enter value for n:");
  scanf("%d",&n);
  printf("Character Pattern is:\n");
  for(i=1;i<=n;i++){
    ch='A';
    for(j=1;j<=i;j++,ch++)
      printf("%c",ch);
    printf("\n");
  }
}
```

Output:

Enter value for n:5
Character Pattern is:
A
AB
ABC
ABCD
ABCDE

---

3. **Multi-Dimensional Array:**
   - ❒ A multi-dimensional array in C contains more than two dimensions (like 3D or 4D arrays). These are rarely used but helpful in special cases like 3D games or image processing.



   - ❒ ***Declaration of Multidimensional Arrays:***
     - ✓ ***Basic Syntax:***

<data_type> <array_name>[<size1>][<size2>][<size3>]...[<sizeN>];

       - data_type: Specifies the type of data (e.g., int, float, char).
       - array_name: Name given to the array.
       - size1, size2, size3, ..., sizeN: Sizes of each dimension.
     - ✓ ***Example for Declaration:***

```
int arr3D[2][3][4];    //3D Array Declaration
float arr4D[2][2][3][5];    //4D Array Declaration
```

   - ❒ ***Initialization of 2-Dimensional Array :***
     - ✓ Array initialization means assigning values to the array elements either at the time of declaration or later using a loop.
     - ✓ ***Compile-Time Initialization*** – initialization at the time of declaration and also called as static initialization.
       - ***Syntax:***

<data_type> <array_name>[<size1>][<size2>]...[<sizeN>]= { {...}, {...}, ... };

       - ***Example:***

```
int arr3D[2][2][3] = {
   { {1, 2, 3}, {4, 5, 6} },
   { {7, 8, 9}, {10, 11, 12} }
};
```

- ✓ ***Run-Time Initialization*** – initialization during program execution and it is also called as dynamic initialization.

```
int arr3D[2][2][3];   // declaration
for(int i = 0; i < 2; i++)
  for(int j = 0; j < 2; j++)
    for(int k = 0; k < 3; k++)
      scanf("%d",&arr3D[i][j][k]);   // Reading 3D array elements
```

❒ ***Accessing Elements of 2D Array in C:***

- ✓ Accessing elements of a multi-dimensional array in C involves specifying indices for each dimension. Each index helps pinpoint the exact location of an element within the array.
- ✓ ***Syntax:***

```
<array_name>[<index1>][<index2>][<index3>]...[<indexN>];
```

  - array_name: Name of the multi-dimensional array.
  - index1, index2, index3,...,indexN: Indices for each dimension, starting from 0.
- ✓ ***Example:***

```
int arr3D[2][2][3] = {
  { {1, 2, 3}, {4, 5, 6} },
  { {7, 8, 9}, {10, 11, 12} }
};
```

To access specific elements:

  - arr3D[0][0][1] → accesses the element 2.
  - arr3D[1][1][2] → accesses the element 12.
- ✓ ***Example Program:***

```
#include <stdio.h>
int main() {
  int arr3D[2][2][3] = {
    { {1, 2, 3}, {4, 5, 6} },
    { {7, 8, 9}, {10, 11, 12} }
  };
  printf("Element at arr3D[0][0][1]: %d\n", arr3D[0][0][1]);
  printf("Element at arr3D[1][1][2]: %d\n", arr3D[1][1][2]);
  return 0;
}
```

Output:

```
Element at arr3D[0][0][1]: 2
Element at arr3D[1][1][2]: 12
```

STRINGS

- String in C programming language is a sequence of characters stored in a character array, ending with a special null character \0 that marks the end of the string.
- It can include letters, numbers, symbols, or spaces—just like a sentence.
- Since C doesn't have a built-in string type like some other languages, strings are managed using arrays and standard library functions.

- ***Declare String in C:***
  - ✓ String declaration in C is done using a character array.
  - ✓ ***Syntax to declare string in C***:

    char <array_name>[<size>];
  - ✓ ***Example of string declaration in C:***

    char name[10];
    - Here, name is a character array that can hold up to 9 characters plus one null character \0. This is just a declaration; no specific string is stored yet. It is like reserving space without filling it.

- ***Initialize String in C:***
  - ✓ After declaring, you can assign a value to the string either directly at the time of declaration or later using functions. This is called string initialization in C programming.
  - ✓ ***Compile-Time Initialization*** – initialization at the time of declaration and also called as static initialization.

```
char str[] = "Hello";
        (or)
char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
        (or)
char city[6] = {'D', 'e', 'l', 'h', 'i', '\0'};
        (or)
char str[10] = "Hello";
```

  - ✓ ***Run-Time Initialization*** – initialization during program execution and it is also called as dynamic initialization.

```
char name[50];
scanf("%s", name);

        (or)

char name[50];
gets(name);

        (or)
```

```
char name[50];
fgets(name, sizeof(name), stdin);
```

- ✓ ***Example Program to read and display a string:***

```
#include<stdio.h>
void main(){
        int str[50];
        printf("Enter a string:");
        scanf("%s",str);
        printf("Entered string is:");
        printf("%s",str);
}
```

Output:

```
Enter a string: programming
Entered string is: programming
```

- ➢ ***Common String Functions in C***
  - ✓ C provides several built-in functions in the string.h header file to perform common operations on strings.
  - ✓ These functions make it easy to manipulate and process strings efficiently.

1. ***strlen() – Get Length of a String***

   Returns the number of characters in a string (excluding the null character)

   Ex:

```
#include <stdio.h>
#include <string.h>
int main() {
   char str[] = "GMRIT";
   int len = strlen(str);
   printf("\n Length of the string=%d", len);
   return 0;
}
```

Output:

```
Length of the string=5
```

2. ***strcpy() – Copy One String to Another***

   Copies the contents of one string into another.

   Ex:

```
#include <stdio.h>
#include <string.h>
int main() {
```

```
    char src[] = "GMRIT";
    char dest[20];
    strcpy(dest, src);
    printf("\n Copied string is : %s", dest);
    return 0;
}
```

Output:

Copied string is: GMRIT

3. ***strcat() – Concatenate Two Strings***

Appends one string to the end of another.

Ex:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str1[20] = "GMRIT ";
    char str2[] = "DU";
    strcat(str1, str2);
    printf("\n Concatenated string is : %s", str1);
    return 0;
}
```

Output:

Concatenated string is: GMRITDU

4. ***strcmp() – Compare Two Strings***

Compares two strings character by character. If both are equal then the function returns 1 otherwise it returns non-zero value.

Ex:

```
#include <stdio.h>
#include <string.h>
int main() {
        int result = strcmp("abc", "abc");
        int result2 = strcmp("abc", "xyz");
        printf("Result 1: %d\n", result);
        printf("Result 2: %d", result2);
        return 0;
}
```

Output:

Result 1: 0

Result 2: -23

5. ***strrev() – Reverse a String***

Reverses the given string.

Ex:

```
#include <stdio.h>
#include <string.h>
int main() {
  char str[] = "GMRIT";
  strrev(str);
  printf("\n Reversed string is: %s", str);
 return 0;
}
```

Output:

Reversed string is: TIRMG

- **String Manipulation without using string functions**
  - ***Program to Find the Length of a String***

```
#include <stdio.h>
#include <string.h>
int main() {
   char str[100];
   int I;
   printf("Enter a string: ");
   gets(str);
   for(i=0;str[i];i++);
   printf("\n Length of given string= %d",i);
   return 0;
}
```

Output:

Enter a string: GMRIT

Length of given string=5

  - ***Program to Copy One String to Another***

```
#include<stdio.h>
void main(){
   char str1[100],str2[100];
   int i;
   printf("Enter your string:");
   gets(str1);
   for(i=0;str1[i];i++)
      str2[i]=str1[i];
   str2[i]='\0';
   printf("\nCopied string is:");
```

```
    puts(str2);
}
```

Output:

```
Enter your string: GMRIT
Copied string is: GMRIT
```

- ***Program to Reverse a String***

```
#include<stdio.h>
void main(){
   char str[100];
   int i,j;
   printf("Enter a string:");
   gets(str);
   printf("Given String is:%s\n",str);
   for(i=0;str[i];i++);
   printf("\nString after reverse:\n");
   for(j=i-1;str[j];j--)
      printf("%c",str[j]);
}
```

Output:

```
Enter a string: GMRIT
Given String is: GMRIT
String after reverse:
TIRMG
```

- ***Program to Count Vowels and Consonants in a String***

```
#include <stdio.h>
#include <ctype.h>
int main() {
   char str[100];
   int vowels = 0, consonants = 0;
   printf("Enter a string: ");
   gets(str);
   for(int i = 0; str[i] != '\0'; i++) {
      char ch = tolower(str[i]);
      if(isalpha(ch)) {
         if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
            vowels++;
         else
            consonants++;
      }
   }
   printf("Vowels: %d\nConsonants: %d", vowels, consonants);
   return 0;
```

```
}
```

Output:

```
Enter a string: GMRIT
Vowels: 1
Consonants: 4
```

- ***Array of Strings:***
  - ✓ An array of strings in C is a collection of multiple strings stored together, typically using a two-dimensional char array or an array of character pointers.
  - ✓ ***Declaration and Initialization of Array of Strings***

```
char fruits[3][10] = {
   "Apple",
   "Banana",
   "Cherry"
};
```

  - ✓ ***Declare Without Initialization***

```
char cities[5][30];  // For 5 city names, each up to 29 characters
```

  - ✓ ***Accessing Strings Using a 2D Character Array***

```
#include <stdio.h>
int main() {
   char cities[3][20] = { "Delhi", "Mumbai", "Jaipur" };
   printf("%s\n", cities[1]);  // Output: Mumbai
   return 0;
}
```

  - ✓ ***Modifying Strings Using a 2D Character Array***

```
#include <stdio.h>
#include <string.h>
int main() {
   char animals[3][20] = { "Dog", "Cat", "Horse" };
   // Change entire string
   strcpy(animals[0], "Lion");
   // Change specific character
   animals[1][0] = 'B';  // Cat → Bat
   printf("%s\n", animals[0]);  // Output: Lion
   printf("%s\n", animals[1]);  // Output: Bat
   return 0;
}
```

  - ✓ ***Input and Output of Array of Strings in C***

```
#include<stdio.h>
void main(){
   char str[5][50];
   int i;
```

```
   printf("Enter 5 strings:");
   for(i=0;i<=4;i++)
      scanf("%s",str[i]);
   printf("Given Strings are:");
   for(i=0;i<=4;i++)
      printf("\n%s",str[i]);
}
```

Output:

```
Enter 5 strings: GMRIT
GMRITDU
AP
SRIKAKULAM
INDIA
Given Strings are:
GMRIT
GMRITDU
AP
SRIKAKULAM
INDIA
```